

硬件迁移 · 确定性计算

迁移不是改 API：从工作负载、数据路径到 Kernel 验收

硬件迁移

数据搬运

GEMV/GEMM

Fallback

岗位能力画像

岗位画像：考察是否能把硬件特性翻译成数据流变化、资源约束和可验证的收益假设。

高频深挖：题目背后的判断

新指令解决了什么？

不要背 feature 名；先指出旧路径的搬运、同步、布局或算力瓶颈。

硬件能力必须映射到 workload 的关键路径。

为什么迁移后仍然慢？

可能是 shape、alignment、layout、tail、寄存器或共享内存预算没有匹配。

迁移前后必须有同口径的 shape × dtype 基线。

确定性怎么兼顾并行？

固定每条归约链的顺序，同时在独立输出链或非归约维上展开并行。

固定顺序不等于关闭全部并行。

fallback 为什么重要？

新路径不覆盖所有 shape；安全回退比‘全量强开’更像工程交付。

选择器和 fallback 是 Kernel 产品的一部分。

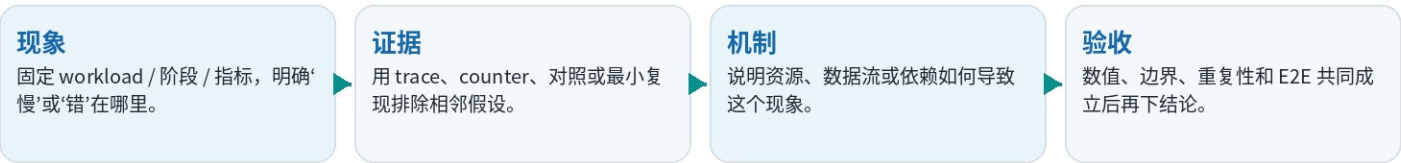
项目叙事的通用深挖链



答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实



一题答透：迁移三层图：工作负载-硬件-Kernel

问题 “如何说明一项异步搬运或矩阵计算特性真的值得接入？”

1. 建基线

按实际 shape、precision、batch 和并行度记录时间线；先确认瓶颈不是上游 launch 或通信。

2. 画数据路

明确 HBM→异步搬运→片上存储→矩阵计算→epilogue 的所有读写和同步点。

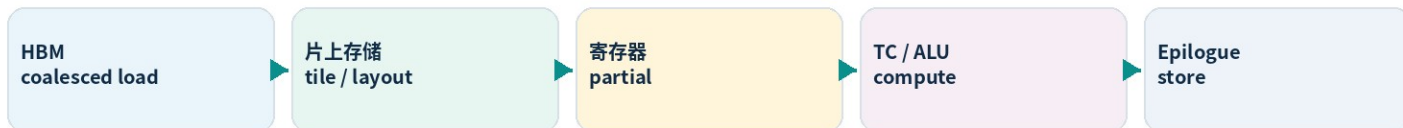
3. 做 dispatch

对 alignment、tail、layout、dtype 建 selector；不满足条件时走成熟库或保守实现。

4. 双验收

数值/确定性与性能同时验收；单 kernel 快不替代端到端验证。

把推理链画出来



常见误区

× 把‘用了新硬件特性’当作性能解释。

× 只测一个漂亮 shape，没有 shape 分布或 fallback。

× 确定性只比较旧/新 bitwise，忽略新实现自身重复运行的一致性。

术语速懂 | 面试怎么答

术语速懂

异步搬运流水

让下一 tile 的传输与当前 tile 的计算重叠；收益取决于布局、对齐、barrier 和资源预算。

确定性归约

浮点加法不满足结合律；固定每条归约链顺序，而不是关闭全部并行。

面试中怎么答

- ① 先说明旧路径卡在搬运、同步还是计算，并给出真实 workload。
- ② 画出 load-buffer-compute 数据路，说明 selector 与 fallback。
- ③ 以数值、重复性、极端 shape 和 E2E 共同验收。

白板 / 手撕：白板题：shape-aware GEMV dispatch

题面 $A[N,K] \times x[K] \rightarrow y[N]$ 。请针对 N/K 极端比例、alignment 和 tail 设计分流。

C++17 · shape-aware GEMV selector

核心实现

```
01 enum class GemvPath { WarpRow, BlockRow, SplitK };
02 GemvPath select_gemv(int n, int k, bool deterministic) {
03     if (n <= 8 && k >= 4096 && !deterministic)
04         return GemvPath::SplitK;
05     if (n >= 128 && k <= 256)
06         return GemvPath::WarpRow;
07     return GemvPath::BlockRow; // tail-safe default
08 }
```

写代码时怎么讲

1. N 大/K 小：warp-per-row 或一个 block 多 row，保证并发。
2. N 小/K 大：split-K 多 block partial，再做二阶段规约；数值约束不允许时说明限制。
3. vectorized load 只在对齐且 tail 安全时启用，其他路径使用 mask。
4. 说清 coalescing、register pressure、block size 和极端 shape 的回退。

实现重点

- 阈值来自 benchmark 校准，不是固定经验。
- Split-K 引入二次归约，要单独说明确定性约束。
- 向量化还必须检查地址对齐与 K 的尾部。

边界、测试与追问

最少要说明的验证

- 阈值来自 benchmark 校准，不是固定经验。
- Split-K 引入二次归约，要单独说明确定性约束。
- 向量化还必须检查地址对齐与 K 的尾部。

面试继续追问

- 什么时候 split-K 会改变确定性？
- 共享内存 padding 与 swizzle 分别解决什么？
- 如何设计 selector 的缓存键？